

Quiz de abertura: o que cada linha imprime?

Anote o seu palpite para cada trecho **antes** de executar. Vamos conferir ao longo da aula.

1 · Python

```
print(-2 ** 2, 2 ** 3 ** 2)
```

2 · Java

```
System.out.println(1 + 2 + "3" + 4 + 5);
```

3 · JavaScript

```
console.log("5" - 2, "5" + 2);
```

4 · C

```
printf("%d\n", 3 > 2 > 1);
```

5 · Python

```
x = 0  
print(x or "padrão", 3 > 2 > 1)
```

6 · JavaScript

```
console.log(0 == "", "" == "0", 0 == "0");
```

As respostas aparecem ao longo da aula · Sebesta Cap. 7

1. Python

a. 4, 512

2. Java

a. Irá somar o que é possível e concatenar com a string “3”

3. JavaScript

a. 552

4. C

a. 1

5. Python

a. True

6. JavaScript

a. True, False, False

Notas do professor após realizarmos os palpites:

Tempo: 5 a 7 min. As duplas anotam os palpites e NÃO executam ainda. Vamos conferindo ao longo da aula.

GABARITO (e onde é explicado):

1) Python: -4 512. O ** tem precedência maior que o menos unário e associa à direita (slide 6).

2) Java: 3345. O + associa à esquerda: 1+2 = 3, depois "3"+"3" = "33", "33"+4 = "334" e "334"+5 = "3345" (slides 6 e 8).

3) JavaScript: 3 52. O - só existe para números, então "5" vira 5. O + prefere concatenar (slide 10).

4) C: 0. 3 > 2 vale 1, e 1 > 1 é falso (0). Os relacionais de C associam à esquerda (slide 11, livro p. 315).

5) Python: padrão True. O 0 conta como falso, e o "or" devolve o segundo operando. Em Python, 3 > 2 > 1 é encadeado: (3 > 2) and (2 > 1) (slides 11 e 12).

6) JavaScript: true false true. O == converte os tipos antes de comparar e NÃO é transitivo (slide 10).

Pergunte quem acertou todas. Normalmente ninguém, e é exatamente essa a motivação da aula.

Expressões aritméticas: questões de projeto

- Quais são as regras de **precedência**?
- Quais são as regras de **associatividade**?
- Em que **ordem os operandos** são avaliados?
- Há restrições a **efeitos colaterais**?
- O usuário pode **sobrecarregar** operadores?
- Que **mistura de tipos** é permitida?

Lisp dispensa a precedência (p. 306)

(+ a (* b c)): notação prefixa, e os parênteses dizem a ordem.

$a + b * c$ com $a=3, b=4, c=5$

da esquerda para a direita

$(3 + 4) * 5 = 35$

da direita para a esquerda

$3 + (4 * 5) = 23$

com precedência (* antes de +)

$3 + 4 * 5 = 23$

APL: sempre da direita para a esquerda (p. 304)

$3 \times 4 + 5 = 27$

Sebesta, 11ª ed. · §7.2 e §7.2.1 · p. 300-305

As regras de precedência são as regras de ordem de cálculo, como por exemplo a precedência matemática que estamos acostumados no dia a dia, como por exemplo, dar prioridade para * e / ao invés de + e -.

$a + b * c$ com $a=3, b=4, c=5$

da esquerda para a direita

$(3 + 4) * 5 = 35$

da direita para a esquerda

$3 + (4 * 5) = 23$

com precedência (* antes de +)

$3 + 4 * 5 = 23$

APL: sempre da direita para a esquerda (p. 304)

$3 \times 4 + 5 = 27$

Existem diferentes maneiras de que o código lê uma operação.

- Da esquerda para a direita
- Da direita para a esquerda
- Com precedência (como realizado em operações matemáticas)
- APL: Sempre da direita para a esquerda.

Precedência e associatividade na prática

Expressão	Python	JavaScript	Java
-2 ** 2	-4	erro de sintaxe	(não há **)
2 ** 3 ** 2	512	512	—
1 + 2 + "3"	TypeError	"33"	"33"
-7 / 2 (inteiros)	-4 (com //)	-3.5	-3
-7 % 2	1	-1	-1

Associatividade (p. 303)
Esquerda: * / % + -
Direita: ** (Python, JS, Ruby, Fortran),
++, --, menos unário
Visual Basic: ^ à esquerda!

Excel
=-2^2 dá 4: lá o menos unário vence o
^.

Na dúvida, use parênteses (p. 305): eles deixam a intenção explícita para o compilador e para o próximo leitor.

Respostas dos palpites realizados anteriormente.

No slide, é realizada uma observação que, em caso de dúvida de como a linguagem realiza a precedência e associatividade das operações, para deixar a operação entre parênteses para deixarmos explícito a ordem para o compilador.

Ordem dos operandos e efeitos colaterais

Exemplo do livro (p. 307)

```
int a = 5;

int fun1(void) {
    a = 17;      /* efeito colateral */
    return 3;
}

int main(void) {
    a = a + fun1(); /* 8 ou 20? */
    printf("a = %d\n", a);
}
```

8
Java
a ordem é garantida: da esquerda para a direita (p. 308)

20
C (gcc 14)
a ordem não é definida pelo padrão

20
Go
a especificação não define a leitura de a em relação à chamada

Transparência referencial (p. 308): sem efeitos colaterais, a ordem deixa de importar, e $f(x) + f(x)$ pode virar $2 * f(x)$.

Nessa página fala sobre os efeitos colaterais que podem acontecer devido a ordem dos operandos. Explica que, a função pode alterar um parâmetro global, em caso de não ter efeitos colaterais, a ordem dos operandos é irrelevante.

Operadores sobrecarregados

- O mesmo símbolo com **vários significados**: + soma inteiros, reais e concatena (Java)
- Risco (p. 309): & em C++ é E bit a bit e "endereço de"
- **Definida pelo usuário** (p. 404): C++, C#, Python, Rust, Kotlin. Java **não** permite

Java

```
System.out.println('a' + 1);           // 98
System.out.println("" + 'a' + 1);     // a1
x.add(y).multiply(tres);               // BigDecimal: sem +
```

Python · sobrecarga com métodos especiais

```
class Vetor:
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __add__(self, outro):           # v1 + v2
        return Vetor(self.x + outro.x,
                      self.y + outro.y)

    def __mul__(self, k):                # v * 3
        return Vetor(self.x * k, self.y * k)

v = Vetor(1, 2) + Vetor(3, 4)          # Vetor(4, 6)
print(v * 3)                           # Vetor(12, 18)
```

Sebesta, 11ª ed. · §7.3 · p. 309-310 e §9.11 · p. 404 · exemplos/03_sobrecarga

Operadores sobrecarregados é quando o mesmo símbolo tem vários significados, um exemplo é a linguagem Java, que o operador + pode concatenar ou somar. Em caso de uso de forma indevida, o usuário pode apresentar valores diferentes do esperado, como concatenar um char com um inteiro ('a' + 1), resultando na conversão do char a + 1.

Conversões de tipos: alargamento, estreitamento e coerção

Java · coerção e cast

```
int grande = 123456789;
float f = grande;           // alargamento implícito
System.out.println((int) f); // 123456792 !

double d = 3.99;
System.out.println((int) d); // 3 (cast)
System.out.println((byte) 200); // -56

byte b = 10, c = 20;
// byte s = b + c; // erro: b + c é int (p. 312)

System.out.println(1 / 2); // 0
System.out.println(1 / 2.0); // 0.5
```

Linguagem	int → real (implícito)	int32 + int64
C	sim	coerção
Java	sim (alargamento)	coerção para long
Python	sim	(um só tipo int)
Go	não : float64(i)	erro de compilação
Rust	não : i as f64	erro de compilação
ML, F# (p. 312)	não	erro de compilação

Coerção esconde erros (p. 312): em d = b * a; com a digitado por engano no lugar de c, o Java converte e compila.

Sebesta, 11ª ed. · §7.4 · p. 310-313 · exemplos/04_conversoes

Aqui, há exemplos mostrados de alargamento em java, onde o número primeiramente é definido como inteiro e depois é atribuído para float, porém por conta do limite de bits, pode haver imprecisão nessa “conversão”, logo os últimos dígitos podem aparecer diferentes do esperado.

Coerção é uma conversão implícita realizada pelo compilador.

Cast é uma conversão explícita realizada pelo programador.

Expressões relacionais e booleanas

C · booleano é inteiro (p. 315)

```
int a = 3, b = 2, c = 1;
printf("%d\n", a > b > c);    // 0 !
// (3 > 2) vale 1, e 1 > 1 é falso

printf("%d\n", (a > b) && (b > c));    // 1

if (-1) printf("verdadeiro\n");    // != 0
```

Python · comparações encadeadas

```
a, b, c = 3, 2, 1
print(a > b > c)    # True: (a > b) and (b > c)

# contam como FALSO em um if:
# 0  0.0  ""  []  {}  None
# contam como VERDADEIRO:
# "0"  [0]  " "
```

Linguagem	O que o if aceita
C, Python, JavaScript	qualquer valor (0, vazio e nulo contam como falso)
Java, C#, Go, Rust	somente booleano: if (n) com n inteiro nem compila

Sebesta, 11ª ed. · §7.5 · p. 313-315 · exemplos/05_booleanas

Expressões relacionais e booleanas: Em C, Python e JavaScript, o if é válido com qualquer valor. Há uma observação que qualquer valor vazio (0, [], {}, none, "") e nulo, são atribuídos a **false**. Em Java, C#, Go, Rust o if aceita apenas expressões booleanas.

Avaliação em curto-circuito

Java · && faz curto-circuito, & não

```
// busca do livro (p. 316)
while (i < lista.length && lista[i] != chave)
    i++;    // && evita lista[lista.length]

String s = null;
s != null && s.length() > 0    // false
s != null & s.length() > 0    // NullPointerException
```

JavaScript · ?. e ??

```
const u = { nome: "Ana", endereco: null };
u.endereco?.cidade    // undefined, sem erro

const qtd = 0;
qtd || 10    // 10 (o 0 se perdeu!)
qtd ?? 10    // 0 (só troca null/undefined)
```

Cuidado com efeitos colaterais (p. 316)

Em (a > b) || (b++ / 3), o b++ só acontece quando a <= b. Em Python, 0 or "padrão" devolve "padrão": and e or devolvem um dos operandos.

Sebesta, 11ª ed. · §7.6 · p. 315-317 · exemplos/05_booleanas

&& realiza curto-circuito, que protege o código de não “explodir” e apresentar erro na execução.

& não realiza curto-circuito, logo não há a proteção para o programa “parar”.

Sentenças de atribuição

C · a atribuição é uma expressão (p. 319-320)

```
sum = count = 0;           // múltiplos alvos
a = b + (c = d / b) - 1;    // exemplo do livro

while ((ch = getchar()) != EOF) { ... }

if (x = y) { ... }          // compila! atribui e testa y
```

Python · atribuição múltipla (p. 320)

```
a, b = b, a                # troca sem temporária
if (n := len(dados)) > 5: ... # morsa (Python 3.8+)
```

Linguagem	Atribuição é expressão?
C, C++, JavaScript	sim: if (x = y) compila
Java, C#	sim, mas o if exige booleano
Python	não, exceto o operador morsa :=
Go	não: nem x++ é expressão
Rust	vale (): if x = 5 não compila

Linguagens funcionais (p. 321)

val (ML) e let (F#, Rust) dão **nome** a um valor, que não muda.

Sebesta, 11ª ed. · §7.7 · p. 317-321 · exemplos/06_atribuicao

Sentenças de atribuição: O slide explica que existem diferentes formas de atribuição

- Simples, composta, unária, expressão, múltipla, funcional;

Estruturas de controle: quantas são necessárias?

Sequência

uma sentença após a outra

```
a = 1;
b = a + 2;
```

Seleção

escolher um caminho

```
if (x > 0) ...
else ...
```

Iteração

repetir enquanto uma condição vale

```
while (x > 0)
  x--;
```

Böhm e Jacopini (1966): sequência, seleção e laço lógico bastam para qualquer algoritmo. O goto não é necessário.

Por que então tantas outras (for, switch, do, foreach)? **Facilidade de escrita.** O limite é a **legibilidade**: quanto maior a linguagem, menos do código alheio você reconhece (p. 329).

Sebesta, 11ª ed. · §8.1 · p. 328-330

O slide explica a importância de entender estruturas de controle como sequência, seleção e iteração.

Seleção de dois caminhos: o "else pendente"

C · a endentação engana (p. 331)

```
int sum = 0, count = 5, result = -1;
if (sum == 0)
    if (count == 0)
        result = 0;
else
    result = 1;           // de qual if é este else?
printf("%d\n", result);  // 1
```

Regra de Java, C, C++ e C#

O else pertence ao if **mais próximo** que ainda não tem else. A endentação é ignorada (exceto em Python e F#).

Chaves obrigatórias eliminam o problema: Perl, Go e Rust (p. 332).

Seletores que devolvem valor

```
let y = if x > 0 { x } else { 2 * x };           // Rust: if é uma expressão (como em F#, p. 334)
y = x if x > 0 else 2 * x                       # Python: expressão condicional (p. 306)
```

Sebesta, 11ª ed. · §8.2.1 · p. 330-334 · exemplos/07_selecao

O Else pendente é atribuído a lugares diferentes dependentes da linguagem, isto é, caso for em C, C++ e C#, ele é atribuído ao if mais próximo, mas se fosse em python, ele seria atribuído ao primeiro if por causa de sua indentação.

Seleção múltipla: do switch de C ao match

C · sem break, o controle "cai" (p. 335)

```
switch (index) {
    case 1:
    case 3: odd += 1;
    case 2:
    case 4: even += 1;
    default: printf("Error\n");
}
// index = 1: odd, even E erro!
```

Java 14+ · switch como expressão

```
String tipo = switch (dia) {
    case SAB, DOM -> "fim de semana";
    case SEG, TER, QUA,
        QUI, SEX -> "dia útil";
}; // sem break, sem cair;
    // exaustivo para enum
```

Rust · match exaustivo

```
match n {
    i32::MIN..=-1 => "negativo",
    0 => "zero",
    1..=9 => "um dígito",
    _ => "vários dígitos",
}
// sem o _ : não compila
```

Go: cada caso termina sozinho, e cair exige fallthrough. C#: cair implicitamente é proibido (p. 336). Python 3.10: match com padrões e guardas.

Sebesta, 11ª ed. · §8.2.2 · p. 334-341 · exemplos/07_selecao

A seleção múltipla é representada de forma como o compilador a lê, de formas diferentes para diferentes linguagens e palavras reservadas. Sendo obrigatório em algumas linguagens a utilização do "break" para que o programa consiga continuar.

Laços controlados por contador

C · o for é um laço lógico disfarçado

```
for (int i = 0; i < 10; i++) {  
    printf("%d ", i);  
    i += 2;           // altera a contagem  
}                    // 0 3 6 9  
  
for (; i <= 100; ) // expressões opcionais  
    soma += i++;
```

Python · o for percorre um iterável

```
for i in range(0, 10):  
    print(i, end=" ")  
    i += 2           # NÃO muda nada  
                    # 0 1 2 3 4 5 6 7 8 9  
  
for i in range(10, 0, -3): # 10 7 4 1  
    ...
```

Questões de projeto (p. 342)

Qual é o tipo e o escopo da variável? Posso alterá-la no corpo? Os parâmetros são avaliados uma vez ou a cada volta?

Sebesta, 11ª ed. · §8.3.1 · p. 342-345 · exemplos/08_iteracao

O escopo da variável é definido dependendo de onde ela foi declarada. Você pode alterá-la no meio do corpo do laço de repetição, o que afetará diretamente no funcionamento da mesma pelo código. Os parâmetros são avaliados a cada volta.

Exercício em duplas: encontre o bug (1/3)

Para cada trecho: (a) qual é a saída? (b) qual regra causa o problema? (c) como corrigir?

1 · JavaScript

```
const precos = [10, 20, 30];  
let total = 0;  
for (const p in precos)  
    total += p;  
console.log("Total: " + total);
```

2 · Python

```
def desconto(preco, d=None):  
    d = d or 10 # padrão: 10%  
    return preco * (100 - d) / 100  
  
print(desconto(200, 0))
```

Arquivos: exemplos/10_exercicio (q1.js ... Q6.java) · execute em onecompiler.com

1. Saída: Total: 0012 // A regra que ocasiona o bug é a forma que é chamada o for, com o **in**. O correto seria utilizar o **for {...} of** para realizar o acesso corretamente.
2. Saída: 180.0 // $d = d \text{ or } 10$. Quando é passado 0 no python, ele entende como **false**, porém no contexto do código, era para ser que não há desconto (**0% de desconto**), uma correção seria colocar uma condicional if para não aplicar o desconto nesse caso.

Exercício: encontre o bug (2/3)

Para cada trecho: **(a)** qual é a saída? **(b)** qual regra causa o problema? **(c)** como corrigir?

3 · Java

```
int dia = 2; String nome = "";
switch (dia) {
    case 1: nome = "domingo";
    case 2: nome = "segunda";
    case 3: nome = "terça";
    default: nome = "inválido";
}
System.out.println(nome);
```

4 · C

```
int saldo = 100, saque = 50;
if (saque > saldo);
    printf("saldo insuficiente\n");
printf("fim\n");
```

Arquivos: exemplos/10_exercicio (q1.js ... Q6.java) · execute em onecompiler.com

1. Saída: inválida // O java faz com que a saída seja inválida pela falta da palavra reservada Break. Sem ela, o código continua sendo percorrido até encontrar o seu fim (neste caso em default). A correção seria a adição da palavra break após a atribuição feita no case.
2. Saída: Saldo Insuficiente e Fim ao mesmo tempo // Isto se deve ao “;” após o if. Isto faz com que o if não seja realizado adequadamente, isto é, ele termina no ; e o compilador lê o printf normalmente de “saldo insuficiente”. A correção seria retirar o “;” do if ou colocar chaves em seus devidos lugares.

Exercício: encontre o bug (3/3)

Para cada trecho: **(a)** qual é a saída? **(b)** qual regra causa o problema? **(c)** como corrigir?

5 · Go

```
x := 10
if x > 5 {
    x := x * 2
    fmt.Println("dentro:", x)
}
fmt.Println("fora:", x)
```

6 · Java

```
int acertos = 7, total = 10;
double taxa = acertos / total * 100;
System.out.println(taxa + "%");
```

Arquivos: exemplos/10_exercicio (q1.js ... Q6.java) · execute em onecompiler.com

1. Saída: Dentro: 20, Fora: 10 // na atribuição curta (:=) o Go associa que o primeiro X dentro do if será uma nova variável local. Então, ele pega o valor de X passado anteriormente e atribui ao segundo X e realiza a expressão normalmente. Apesar

disso, o X declarado fora do If mantém seu valor devido a shadowing. uma correção seria trocar := por =.

2. Saída: 0% // Acontece isso por conta que são declarados como inteiro, e ao realizar a divisão (7/10) a linguagem apresenta o número 0 (sem o ponto flutuante. Uma correção seria fazer o cast explícito (double) $\text{acertos} * 100 / \text{total}$